

A Small Verifier That Learns From Outcomes:

Judging coding-agent runs locally with Gemma 4, and separating issue difficulty from run quality

John Soliva

D1A project, <https://github.com/jonpol01/d1a>

DRAFT — October 5, 2026

Abstract

A coding agent cannot see the hidden tests that decide whether its patch resolves an issue. A verifier that estimates $P(\text{resolved})$ can pick among several runs, filter runs for training, and learn from outcomes as they arrive. We study a small one: D1A-E4B (Gemma 4 E4B, a LoRA adapter and a pointer head), which reads the issue, the agent’s patch and the end of its run, and answers in one forward pass in a median 1.5s on a laptop. Our central observation is that the usual metric hides what matters. On 921 public agent runs from 83 unseen repositories, *knowing how other runs of the same issue fared* already scores AUROC 0.958, so global AUROC mostly measures issue difficulty. Ranking runs *of the same issue* (within-issue AUROC) is the harder test, and on it no scorer we tried is clearly better than another: on 119 test issues with mixed outcomes, cheap trajectory heuristics reach 0.622, D1A 0.616 zero-shot and 0.642 after a 2.5-hour LoRA on a laptop (+0.020 over the heuristics, 95% CI $[-0.053, +0.093]$). The same confound inflates learned verifiers evaluated on random splits of runs: on 32,340 runs, a text verifier scores AUROC 0.946 under a random split and 0.736 on unseen repositories, and its within-issue AUROC falls from 0.643 to 0.493. Finally, D1A learns from outcomes through a loop that recalibrates, retrains and promotes a candidate only through a statistical gate; over three rounds of 300 outcomes, recalibration cut the calibration error from 0.068 to 0.017–0.036, while no retrained candidate was clearly better and the gate kept the current model every time. All data is public and all code is open.

1 Introduction

LLM coding agents edit repositories in long runs of tool calls and then submit a patch, without knowing whether the hidden tests will pass. This is the setting of the Gemma 4 Developer Agent challenge, where an agent runs offline on a fixed Gemma 4 model under a time budget per task: a small verifier on the same hardware is what such an agent can afford to consult. A verifier’s value lies in decisions *about a given issue*: which of several runs to submit, which runs to keep as training data, when to try again. These decisions compare runs of the same issue, yet verifiers are usually evaluated by global AUROC over runs of many issues, which also rewards recognising which issues are easy.

Verifiers for coding agents already exist (Section 2); we do not claim a new kind of verifier. Our contributions are:

1. **Separating issue difficulty from run quality.** On held-out repositories, an issue prior alone reaches global AUROC 0.958. We report within-issue AUROC next to global AUROC and find that within issues a small decision model and cheap heuristics are indistinguishable (Section 3).

2. **Split leakage, measured.** On the same runs, random-run splits let a learned verifier score by recognising issues; grouping by issue removes the inflation (Section 4).
3. **A local verifier that learns from outcomes**, with a statistical promotion gate, released as a library (`d1a.feedback`) (Section 5).

2 Related Work

Verifiers for software-engineering agents. SWE-Gym trains verifiers on agent trajectories and uses them for inference-time scaling, selecting among sampled trajectories [1]. SWE-RM is a 30B execution-free reward model that treats classification accuracy and calibration as crucial for reinforcement learning [2]. Agentic Rubrics derive contextual rubrics as a verification signal [4]. Closest to us, a rubric-supervised critic is learned from sparse real-world outcomes, jointly predicting 24 behavioural rubrics and success, and improves best-of-8 reranking over random on the “rerankable” subset of trajectories [3]; restricting evaluation to issues with mixed outcomes is thus established. What we add is the decomposition itself: how much of pooled AUROC an issue prior explains, how heuristics and a small decision model differ within issues, and how random splits inflate learned verifiers.

Early failure prediction. FailFast-RestartSmart trains a 0.6B monitor on trajectory prefixes and restarts runs predicted to fail [5]; EarlyEval predicts outcomes early with gradient-boosted classifiers [6]. Our anytime experiment is a weaker instance of this idea and is reported as a contrast.

Self-improvement with gates. Audited skill-graph self-improvement promotes a candidate only after verifier-backed replay [7], and self-authored verification is shown to be unreliable, motivating an external acceptance gate [8]. Verification also has to keep up with the policy it judges [9]. Our gate is external in the same sense, driven by real outcomes, and decides with a bootstrap interval over repositories.

3 The Verifier

Model. D1A [12] is a decision model built on Kev [11]: a causal LM backbone with a LoRA adapter runs one prefill over a document and typed questions under a block-causal mask, and a pointer head scores each option against the question’s decision token. The backbone is Gemma 4 E4B. D1A-E4B v0.4, released before this work, was trained on general decisions and pull-request labelling, never on agent runs.

Input. The document holds the issue (up to 3,000 characters), the agent’s final patch (up to 6,000) and the last steps of its run (up to 3,000; each step up to 800). The question asks whether the patch correctly fixes the issue, so that the repository’s tests for it would pass. Inputs are 1.5k tokens at the median and 5.5k at most.

Data. nebius/SWE-agent-trajectories (CC-BY-4.0) holds 80,036 SWE-agent runs on 3,591 issues, each with `target` (resolved by the hidden tests), the trajectory and the patch; the test logs are never read. Repositories are assigned to train (70%), dev (10%) and test (20%) by a hash of the name. We prepared 2,000 training runs from 377 repositories (at most four per issue, successes

first); dev holds 1,142 runs and test 921 runs from 83 repositories (up to 8 per issue; 12.4% resolved; 23 of 123 test issues have both outcomes).

Training and calibration. One LoRA run continues D1A-E4B v0.4 for 100 optimizer steps of 8 runs (800 of the prepared runs; learning rate $5 \cdot 10^{-5}$), 2.5 h on an Apple M1 Max at no cost. Platt scaling on the logit is fitted on dev only. The heuristics baseline is a logistic regression over six trajectory features (patch size, a clean “submitted” exit, number of steps, error observations, printed success, test-file edits), fitted on training-split runs. The “+ heuristics” combinations refit the heuristic weights jointly with D1A’s logit on dev; refitting the heuristics alone on dev does not improve them.

	AUROC	within-issue (23 issues)	ECE	Brier
Issue prior (other runs of the same issue)	0.958	–	–	–
Heuristics (logistic regression)	0.768	0.536	0.019	0.100
D1A-E4B v0.4 zero-shot, recalibrated	0.742	0.617	0.043	0.103
D1A-E4B + verifier LoRA (r1), recalibrated	0.807	0.661	0.039	0.097
+ heuristics	0.819	0.667	0.037	0.094

Table 1: 921 test runs from 83 repositories unseen in training (base-rate Brier 0.108). Within-issue AUROC averages over the 23 issues with both outcomes. The issue prior is a reference, not a usable verifier: it needs other runs of the same issue with known outcomes. Global AUROC differences against the heuristics (95% CI, bootstrap over repositories): r1 +0.039 [−0.030, +0.109], heuristics + r1 +0.051 [+0.002, +0.101]. Zero-shot D1A before recalibration: ECE 0.632.

Issue difficulty dominates global AUROC. Most issues are resolved by all or none of their runs, so a scorer that recognises the issue ranks runs well globally without judging any run. Within issues the picture changes (Table 1): on our 23 mixed test issues the heuristics reach 0.536, D1A 0.617 zero-shot and 0.661 after training. Those 23 issues are too few: on 119 mixed-outcome issues from the test repositories of all 80,036 runs (952 runs, up to 8 per issue, each issue keeping at least one success and one failure), the gap closes (Table 2). Within issues, a 4B-class decision model reading the issue, the patch and the end of the run does not judge runs better than six trajectory features.

Scorer	within-issue AUROC	global AUROC (these runs)
Heuristics (logistic regression)	0.622 [0.569, 0.676]	0.659
D1A-E4B v0.4 zero-shot	0.616 [0.562, 0.666]	0.645
D1A-E4B + verifier LoRA (r1)	0.642 [0.587, 0.698]	0.691

Table 2: 119 test issues with mixed outcomes (952 runs, 42% resolved by construction); 95% intervals from a bootstrap over issues. r1 minus heuristics: +0.020 [−0.053, +0.093]; zero-shot minus heuristics: −0.006 [−0.069, +0.061].

Picking one of several runs. Picking one of 8 recorded runs per issue (97 issues, 23 with mixed outcomes; bootstrap over issues), compared with a random pick (14.7% resolved): zero-shot D1A +5.9 points [+2.2, +10.2], heuristics + r1 +4.9 [+0.5, +9.3], r1 alone +2.8 [−1.0, +6.6], heuristics

$-1.3[-5.0, +2.7]$; an oracle reaches 26.8%. At $k = 2$ and $k = 4$ the gains are smaller and not significant. Training did not improve selection over the zero-shot model here.

Filtering training data. Keeping the 10% of test runs that r1 trusts most gives 40.2% truly resolved runs (37), against 22.8% for the heuristics (21), 19.6% for the agent’s own clean exit and 15.2% at random.

Calibration, honestly. After recalibration all scorers are reasonably calibrated, and the heuristics are best (ECE 0.019). No scorer is confident enough to decide “submit or keep working” on its own: the highest precision any threshold reaches with at least 10 runs submitted is 41% for r1 (9.0% of runs) and 65% for heuristics + r1 (1.8% of runs). We therefore recommend the probability for comparing runs and filtering data, not as a stand-alone stop signal.

Anytime verification is weak. Scoring an unfinished run at its tenth action (patch absent, final exit hidden), the heuristics fall below chance (AUROC 0.40) and zero-shot D1A reaches 0.66, which is +0.16 over chance. Replaying stop-and-restart policies on recorded runs under an action budget, the verifier gains at most 1.3 points (12.8% vs 11.5% for never stopping at a 90-action budget; oracle 15.9%), far less than FailFast-RestartSmart reports with a dedicated prefix monitor [5].

Latency. Median 1.48 s and p95 2.79 s per run (MLX, 8-bit, M1 Max).

4 Split Leakage

The issue-difficulty confound also inflates learned verifiers when runs of one issue fall on both sides of a split. We measure this on the same runs under three 5-fold protocols (random runs; grouped by issue; grouped by repository) with four verifiers refitted per fold: an issue prior from the training folds (pure leakage by construction), the heuristics, a TF-IDF model of the issue and patch text, and both together.

Verifier	global AUROC			within-issue AUROC		
	random	by issue	by repo	random	by issue	by repo
Issue prior	0.947	0.480	0.452	–	–	–
Heuristics	0.754	0.752	0.750	0.647	0.645	0.647
Text (TF-IDF)	0.946	0.749	0.736	0.493	0.646	0.643
Heuristics + text	0.944	0.792	0.787	0.538	0.665	0.661

Table 3: Out-of-fold AUROC under three 5-fold splits of the same 32,340 runs (3,591 issues, 1,276 repositories, at most 10 runs per issue, 10.6% resolved; within-issue over the 662 issues with both outcomes; `runs/swe-verifier/leakage/result.json`). Inflation of global AUROC over the by-repository split (95% CI, bootstrap over repositories): text +0.210 [+0.192, +0.229], heuristics + text +0.158 [+0.144, +0.174], heuristics +0.005 [+0.001, +0.009]; by issue: text +0.013 [−0.004, +0.027], heuristics + text +0.005 [−0.006, +0.016].

Under a random split the text model’s global AUROC is 0.946, yet its within-issue AUROC falls to 0.493: when issues leak, it learns to recognise them *instead of* judging runs, and loses the within-issue discrimination it has when issues cannot leak (0.643). Grouping by issue removes the inflation, and grouping by repository adds little beyond that in this data. Evaluations of learned

verifiers should group by issue at least, and report within-issue discrimination. We have not checked whether any published verifier result used a random split.

5 Learning From Outcomes

d1a.feedback logs each decision with the model version that made it, records the outcome when it arrives, and improves the model in three steps, cheapest first: (1) an outcome calibrator refits each yes/no probability on the outcomes (Platt scaling [10], which also corrects a shifted base rate); (2) a small LoRA update trains on the resolved decisions; (3) a promotion gate replaces the current model only if the candidate’s held-out log loss is lower with a 95% bootstrap interval over repositories entirely below zero, and no frozen evaluation suite regressed by more than one point.

Rounds. Starting from r1, batches of 300 unseen runs arrive with their outcomes. Each round, 80% of the batch (by repository) becomes training records and 20% a calibration slice; each model is recalibrated only on slices it never trained on; the gate decides on dev repositories; the served model is measured on test repositories.

Round	outcomes	test AUROC	test ECE	gate
0 (r1, no feedback)	0	0.808	0.068	–
1	300	0.808	0.036	rejected: log loss +0.026 [+0.016, +0.058]
2	600	0.808	0.017	rejected: +0.010 [−0.004, +0.050]
3	900	0.808	0.031	rejected: −0.007 [−0.016, +0.014]

Table 4: The served verifier after each round on test repositories (four runs per issue). Recalibration on the held-out calibration slices (77, 147 and 210 outcomes) cut the calibration error; the retrained candidates (30 steps each on the 223, 453 and 690 training records so far) were worse (round 1) or not clearly better (rounds 2–3), and the gate kept the current model, so the served AUROC did not change.

Three rejections do not show that retraining cannot help, only that 30-step updates on a few hundred outcomes did not, and that the gate stops what is not clearly better. The gate’s power is limited by the dev set (about 7% resolved): only a sizable log-loss improvement can clear a 95% interval. Larger outcome batches, several candidate settings per round, and a measured minimum detectable effect are the next steps.

Adapting to a new agent. A verifier trained on one agent meets new agents as they appear, and a fixed reward function falls behind as the policy improves [9]. [\[pending: Gemma-4-31B agent runs from the challenge \(repeated runs per task\): within-issue AUROC and calibration after \$N\$ outcomes from the new agent, recalibration alone vs recalibration + LoRA, with gated promotion.\]](#)

6 Limitations

One agent family (SWE-agent with Llama models) carries the results so far; the Gemma 4 agent study is pending. Only 23 test issues have mixed outcomes, so within-issue and best-of- k estimates are noisy. The verifier sees truncated inputs. The challenge provides per-repository code graphs and embeddings; this verifier does not use them, because it judges a finished run from its issue, patch and trajectory, and in a separate study of ours a D1A localiser over the provided graphs did

not beat BM25 at finding the code to change. Under 5-fold cross-validation on a separate sample of 600 runs, zero-shot D1A appeared to add +0.069 global AUROC to the heuristics; on the fixed test split it added nothing (−0.003). Both were grouped by repository, so this is a gain that did not replicate, not leakage.

7 Reproducibility

All code is in <https://github.com/jonpol01/d1a> (recipes/swe-verifier: `zero_shot.py`, `to_records.py`, `evaluate.py`, `best_of_k.py`, `budget_sim.py`, `leakage.py`, `self_improve.py`; library `d1a.feedback`). Data is public (CC-BY-4.0, MIT). Training and evaluation ran on one Apple M1 Max laptop at no cost.

References

- [1] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, Y. Zhang. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv:2412.21139, 2024.
- [2] K. Shum, B. Hui, J. Chen, L. Zhang, X. W., J. Yang, Y. Huang, J. Lin, J. He. SWE-RM: Execution-free Feedback for Software Engineering Agents. arXiv:2512.21919, 2025. (The author “X. W.” is listed in this form on arXiv.)
- [3] X. Wang, V. Chen, H. Ji, G. Neubig. A Rubric-Supervised Critic from Sparse Real-World Outcomes. arXiv:2603.03800, 2026.
- [4] M. Raghavendra, A. Gunjal, B. Liu, Y. He. Agentic Rubrics as Contextual Verifiers for SWE Agents. arXiv:2601.04171, 2026.
- [5] C. Wang, Y. Lyu, J. He, Z. Yang, C. Zhong, Y. Harel, D. Lo. Fail-Fast, Restart-Smart: Early Failure Prediction and Restart for SWE Agentic Tasks. arXiv:2608.03222, 2026.
- [6] Y. Shi, Z. Sun, J. Dong, C. Wan, D. Lo, X. Gu. EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction. arXiv:2609.02783, 2026.
- [7] K. Huang, J. Huang. Audited Skill-Graph Self-Improvement for Agentic LLMs via Verifiable Rewards, Experience Synthesis, and Continual Memory. arXiv:2512.23760, 2025.
- [8] D. Guo, C. Cao, F. Yuan, Y. Wang, Y. Wang, D. Wang. Self-Authored Verification Is Unreliable in Heuristic Self-Improving Agents. arXiv:2607.24300, 2026.
- [9] B. Wang, C. Zhang, D. Liu, J. Zhang, J. Chen, M. Li, M. Chen, R. Fang, S. Zhang, X. Wang, Y. Jing, Z. Ma, Z. Cui. The Verification Horizon: No Silver Bullet for Coding Agent Rewards. arXiv:2606.26300, 2026.
- [10] J. C. Platt. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*, MIT Press, 1999.
- [11] J. Palmer. Kev. <https://github.com/jaredpalmer/kev> (Apache-2.0).
- [12] J. Soliva. D1A. <https://github.com/jonpol01/d1a> (Apache-2.0).